CO_2 - AT3

SCENARIO BASED TASK

NAME : REKHA.SK

REG NO: 192511023

COURSE CODE : CSA1407

COURSE NAME: COMPILER DESIGN

DATE: 07/08/2026

FACULTY: Dr. N. Deepa.

1) Parsing Table Conflict in LL(1)

The syntax analyzer uses FIRST and FOLLOW sets to construct an LL(1) parsing table. For each production, entries are placed in the table based on the FIRST set of the production. If the production can derive ε - epsilon, the FOLLOW set of the non-terminal is also used.

Multiple entries appear in the same parsing table cell, indicating that the parser cannot uniquely decide which production to apply. This conflict usually occurs because:

- The grammer is left recursion, or
- Two or more productions have common prefixes, causing overlapping FIRST sets.

As a result, the parser loses its deterministic nature, since one input symbol corresponds to more than one production. Therefore, the grammer is not LL(1).

To make the grammer for LL(1) parsing:

- Eliminate left recursion.

• Apply left factoring to remove common prefixes.

• Ensure that the FIRST and FOLLOW sets do not produce conflicts.

After these modifications, each parsing table entry will contain only one production, making the grammer suitable for deterministic LL(1) parsing.

2) Incorrect Parse Tree Generation.

The syntax analyzer constructs a parse tree according to the grammer rules. The parse tree determines how operands and operators are grouped during expression evaluation.

For the expression:

id + id - id

if the grammer does not specify associativity, the expression may be grouped incorrectly, producing wrong evaluation results.

The problem is related to operator associativity. Since + and - are left-recursion operators, the correct grouping should be:

(id + id) - id

rather than

id + (id - id)

The grammer directly influences the structure of the parse tree. An ambiguous grammer may allow multiple parse tree for the same expression.

The grammer should explicitly enforce left associativity, for example:

E -> E+T / E-T | T

T -> id

or use parser precedence rules so that only one correct parse tree is generated, ensuring accurate evaluations.

3) Invalid Expression Handling,

The lexical analyzer correctly recognizes the tokens:

a+ = b

Although each tokens is valid individually, the sequence does not follow the grammer rules for arithmetic expressions.

After the '+' operator, the grammer expects an operand, but another operator ('*') appears, instead. Therefore, the syntax analyzer cannot construct a valid parse tree and reports a syntax error.

In predictive parsing, the parser checks the parsing table and finds no valid production for the current input symbol, leading to an error.

In shift-reduce parsing, the parser cannot perform either a valid shift or reduction according to the grammer, so it reports a syntax error.

To improve error handling:

- Use meaningful syntax error messages.
- Apply error recovery techniques such as panic mode recovery or phrase-level recovery.
- Continue parsing after recovery whenever possible.

These techniques improve error detection while maintaing parsing clarity.

4) Ambiguity in Conditional Statements.

The grammar:

S → if E then S
| if E then S else S
| a

the syntax analyzer encounters ambiguity because it cannot determine whether an else belongs to the nearest if or an earlier if.

This is known as Dangling Else Problem.

Due to this ambiguity:

- Multiple parse trees are possible.
- Parsing table conflicts occur.
- Deterministic parsing becomes impossible.

The ambiguity can be removed by rewriting the grammer using matched and unmatched statements, for example:

Matched → if E then Matched else Matched | a

Unmatched → if E then S
| if E then Matched else Unmatched.

Alternatively, use a parser strategy that always associates else with the nearest unmatched if,

which is the standard rule used by most programming languages.

5) Operator Precedence Conflicts

Consider the grammer:

$E \rightarrow E + E$

$\mid E * E$

$\mid id$

This grammer is ambignous because it does not specify operator precedure or associativity.

For the expression:

$id + id * id$.

the parser may generate different parse tree. such as:

. $(id + id) * id$

. $id + (id * id)$

The correct interpretation is:

$id + (id * id)$

because multiplication has higher precedence than addition.

The ambiguity causes incorrect evaluation.

Modify the grammer to explicity define precedence and associativity, for example:

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Here:

- * has higher precedence than +.
- Both operators follow left associativity.

Alternatively, use parser precedence and associativity rules in LR parsers to resolve conflicts.